

Git 進階

刪除不必要的分支

假設我們 feature 分支還會持續開發新功能，所以可以暫時將它留著，讓後續還能繼續切換到 feature 分支繼續開發。

如果你確定用不到的話，可以用 `git branch -d feature` 刪除該分支。

```
G:\git-merge-demo>git branch -d feature  
Deleted branch feature (was 09dc26e).
```

在 Git 裡，只要沒有執行過「合併」的分支，都不能用上述指令進行刪除，必須改用 `git branch -D feature` 才能刪除該分支。

救回誤刪的分支

不過，如果你不小心「誤刪」該分支 (feature) 的話，則必須用以下步驟救回分支，稍微麻煩一點：

1. 先利用 `git reflog` 找出該分支 (feature) 最後一個版本的 **object id** (也就是 **SHA1** 格式的物件絕對名稱)
2. 執行 `git branch feature <SHA1>` 即可

如下圖示，最終還是能把誤刪的 feature 分支給救回：

```

G:\git-merge-demo>git branch
* master

G:\git-merge-demo>git reflog
47ad369 HEAD@{0}: merge feature: Merge made by the 'recursive' strategy.
c47287e HEAD@{1}: commit: Update a.txt: append NEW LINE
e5c508f HEAD@{2}: checkout: moving from feature to master
09dc26e HEAD@{3}: checkout: moving from master to feature
e5c508f HEAD@{4}: checkout: moving from feature to master
09dc26e HEAD@{5}: checkout: moving from master to feature
e5c508f HEAD@{6}: checkout: moving from feature to master
09dc26e HEAD@{7}: commit: Add b.txt
e5c508f HEAD@{8}: checkout: moving from master to feature
e5c508f HEAD@{9}: commit: Update a.txt!
adb712e HEAD@{10}: commit (initial): Initial commit (a.txt created)

G:\git-merge-demo>git branch feature 09dc26e

G:\git-merge-demo>git branch
feature
* master

G:\git-merge-demo>git log feature
commit c47287e4dc78f454584e9ad7d00a9a69b927e96e
Author: Will <xxx@gmail.com>
Date:   Wed Oct 16 20:13:41 2013 +0800

    Update a.txt: append NEW LINE

commit e5c508f0b2d91e10063ede2b3f684385536d3c15
Author: Will <xxx@gmail.com>
Date:   Wed Oct 16 19:43:58 2013 +0800

    Update a.txt!

commit adb712e4ef2317e5823a3560a356e0863c3d2658
Author: Will <xxx@gmail.com>
Date:   Wed Oct 16 19:43:53 2013 +0800

    Initial commit (a.txt created)

G:\git-merge-demo>_

```

解決衝突狀態的方法

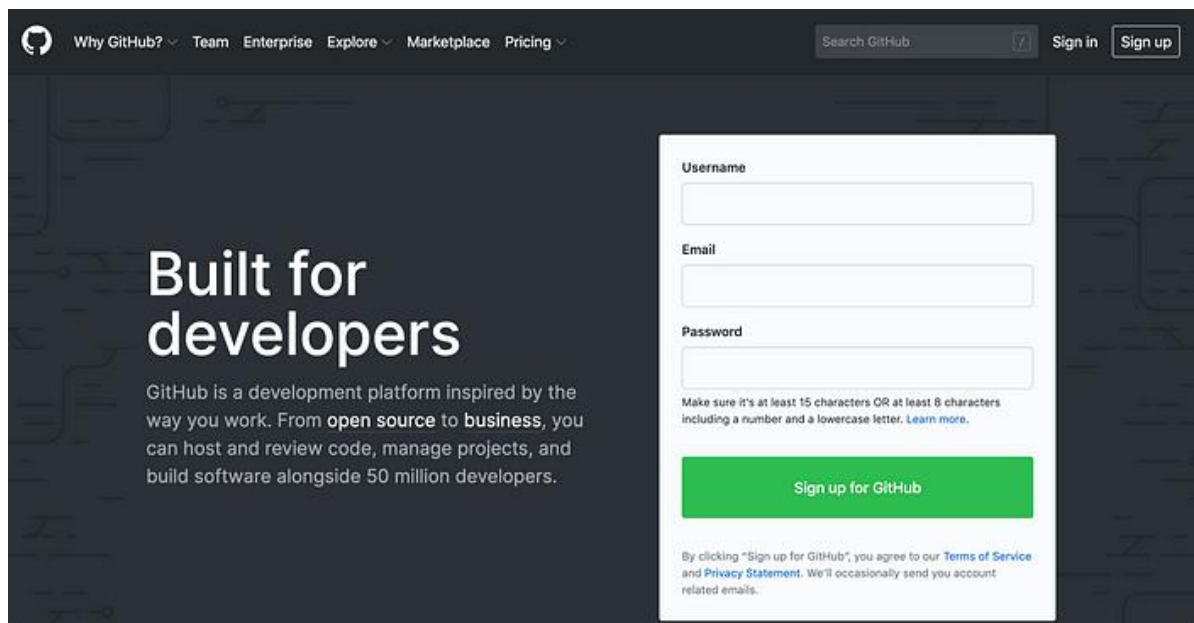
我先來說說「解決衝突狀態」的方法，其實很簡單，直接輸入 `git add .` 把目前工作目錄的狀態加入到「索引」之中，就可以取消這些 Unmerged 檔案的衝突狀態。

如果你很懶，遇到衝突什麼都不改，直接執行 `git add .` 再加上 `git commit` 的話，確實會「解決衝突」，但所 `commit` 的版本將會包含這些衝突的表示文字，如果你身處一個開發團隊，肯定會被罵翻到臭頭，請你千萬不要這麼做！

只要執行 `git reset --hard ORIG_HEAD` 就可以回復到上一版，然後再重新合併一次引發相同的衝突。

註冊

進入 GitHub 官方網站：<https://github.com/>

The image shows the GitHub sign-up page. On the left, there's a dark blue background with the text "Built for developers" in large white font. Below it, a smaller white text describes GitHub as a development platform. On the right, there's a white sign-up form with fields for "Username", "Email", and "Password". The "Password" field has a note about password requirements. Below the fields is a green "Sign up for GitHub" button. At the bottom of the form, there's a small disclaimer about agreeing to terms and privacy.

Join GitHub

Create your account

Username *

Email address *

Password *

Make sure it's at least 15 characters OR at least 8 characters including a number and a lowercase letter. [Learn more.](#)

Email preferences

☐ Send me occasional product updates, announcements, and offers.

Verify your account

請解開這謎題，
好讓我們知道你是真人

驗證

Create account

 Search or jump to... Pull requests Issues Marketplace Explore

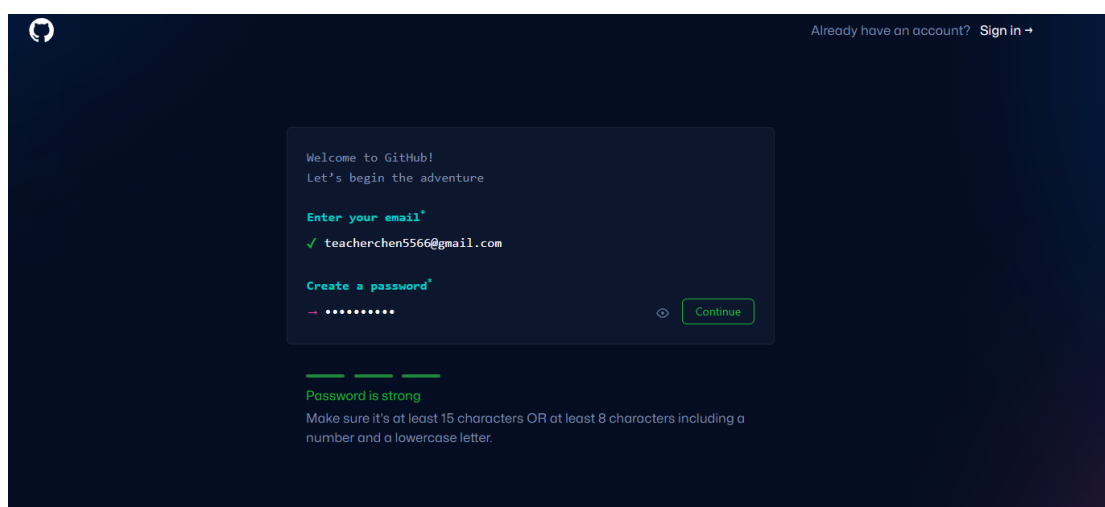
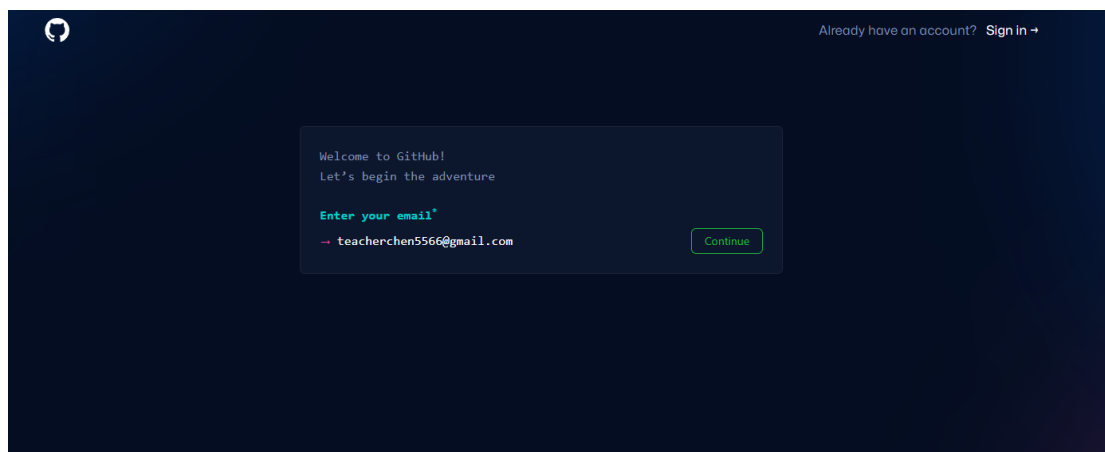
Selected plan: Free

Welcome to GitHub

Woohool You've joined millions of developers who are doing their best work on GitHub. Tell us what you're interested in. We'll help you get there.

What kind of work do you do, mainly?

Software Engineer I write code	Student I go to school
Product Manager I write specs	UX & Design I draw interfaces
Data & Analytics I write queries	Marketing & Sales I look at charts





Sign in to GitHub

Your account was created successfully. Please sign in to continue



Username or email address

Password

[Forgot password?](#)

Sign in

[Sign in with a passkey](#)

New to GitHub? [Create an account](#)

This will help us guide you to the tools that are best suited for your projects.

How would you describe yourself?

☒ Student

☐ Teacher

☐ Working developer

☐ Other

How many team members will be working with you?

☒ Just me

☐ 2-5

☐ 5-10

☐ 10-20

☐ 20-50

☐ 50+

Continue

What are the top 2 things you want to do with GitHub?

☒ Start a new project

☐ Start or expand my business

☒ Use GitHub Copilot

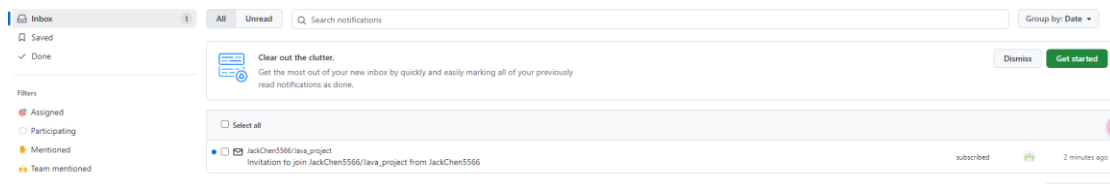
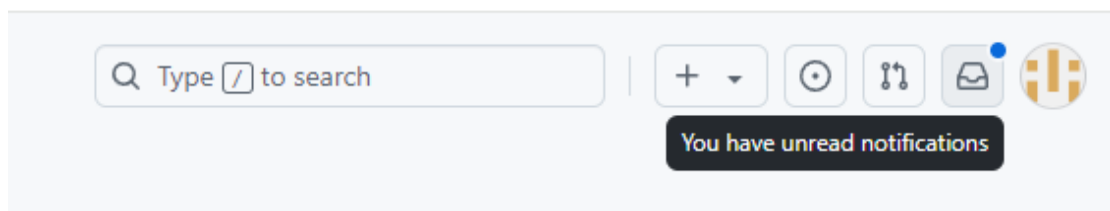
☐ Connect with other developers

☐ Contribute to Open Source projects

Select up to 2 options

Continue

邀請確認





[JackChen5566](#) invited you to collaborate

Accept invitation

Decline

🔒 Owners of Java_project will be able to see:

- Your public profile information
- [Certain activity](#) within this repository
- Country of request origin
- Your access level for this repository
- Your IP address

Is this user sending spam or malicious content?

[Block JackChen5566](#)

Download:

<https://github.com/apps/desktop>

與遠端儲存庫有關的指令

- `git clone`

將遠端儲存庫複製到本地，並建立工作目錄與本地儲存庫 (就是 `.git` 資料夾)

- `git pull`

將遠端儲存庫的最新版下載回來，下載的內容包含完整的物件儲存庫 (object storage)。並且將遠端分支合併到本地分支。(將 origin/master 遠端分支合併到 master 本地分支)

所以一個 git pull 動作，完全相等於以下兩段指令：

```
git fetch git merge origin/master
```

- git push

將本地儲存庫中目前分支的所有相關物件推送到遠端儲存庫中。

- git fetch

將遠端儲存庫的最新版下載回來，下載的內容包含完整的物件儲存庫 (object storage)。這個命令不包含「合併」分支的動作。

- git ls-remote

顯示特定遠端儲存庫的參照名稱。包含遠端分支與遠端標籤。

關於追蹤分支的概念

我們先前學習過關於「分支」的概念，不過僅限於「本地分支」，今天我們多出了個「遠端分支」，事情就相對複雜一些。

基本上，當我們的版本控管流程加上了遠端儲存庫之後，原本的分支就可以被拆成四種不同的概念：

1. 遠端追蹤分支

這個分支位於遠端，目的是用來追蹤分支的變化情形。通常遠端分支你是存取不到的。

2. 本地追蹤分支

當你執行 git clone 複製一個遠端容器回來之後，所有遠端追蹤分支會被下載回來，並且相對應的建立起一個同名的本地追蹤分支。

我們以複製 jQuery 在 GitHub 上的專案為例，透過 git clone <https://github.com/jquery/jquery.git> 下載回來後，執行 git branch -a 指令，

顯示出所有「本地分支」與「本地追蹤分支」。「本地追蹤分支」就是如下圖紅字的部分：

```
C:\>git clone https://github.com/jquery/jquery.git
Cloning into 'jquery'...
remote: Counting objects: 30855, done.
remote: Compressing objects: 100% (9074/9074), done.
remote: Total 30855 (delta 22610), reused 29212 (delta 21150)
Receiving objects: 100% (30855/30855), 15.08 MiB | 685.00 KiB/s, done.
Resolving deltas: 100% (22610/22610), done.
Checking out files: 100% (208/208), done.

C:\>cd jquery

C:\jquery>git branch -a
* master
  remotes/origin/#13388
  remotes/origin/1.8-stable
  remotes/origin/1.9-stable
  remotes/origin/1.x-master
  remotes/origin/HEAD -> origin/master
  remotes/origin/ajax-script-2.0
  remotes/origin/ajax-unit
  remotes/origin/master
```

3. 本地分支

在透過 `git branch` 指令執行時所顯示的分支，就是所謂的「本地分支」，這些分支存在於本地端，而這些分支又常被稱為 **主題分支 (Topic Branch)** 或 **開發分支 (Development Branch)**，就是因為這些分支預設不會被推送到遠端儲存庫，主要用來做開發用途。

4. 遠端分支

顧名思義，遠端分支就是在遠端儲存庫中的分支，如此而已。如果你用 GitHub 的話，你是無法存取遠端分支的。

雖然說「概念上」可以分為這四類，但其實這些分支就只是**參照名稱**而已，而這裡的「追蹤分支」主要就是用來跟遠端的分支做對應，你不應該直接在這些分支上建立版本（雖然你還是可以這麼做，但強烈不建議亂搞），而是把這些「本地追蹤分支」視為是一種「唯讀」的分支。

Clone 專案

https://github.com/JackChen5566/Java_project